

Add a conditional `noexcept` specification to `std::apply`

- - Abstract
 - Revision history
 - Discussion
 - Impact on the standard
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes to add a `noexcept`-specification to `std::apply`.

Revision history

R1

Added missing revision to header `<tuple>` synopsis. Aligned proposed changes with the latest draft.

R0

Initial revision.

Discussion

With the introduction of the C++23 `zip` family, `apply` is making a comeback and appeared in a lot of its function implementations.

For example, in [\[range.zip.transform.iterator\]](#), `zip_transform_view::iterator::operator*()`'s equivalent *Effect* is defined as:

```
return apply([&](const auto&... iters) -> decltype(auto) {
    return invoke(*parent_->fun_, *iters...);
}, inner_.current_);
```

which uses `apply` to extract the elements of the iterator `tuple` and forwards them into the callable.

However, strictly speaking, this `operator*()` should not be done through `apply`. The reason is that it still has a `noexcept` (see below) specification which is equivalent to `noexcept(invoke(*parent_->fun_, *std::get<Is>(inner_.current_)...))`, where `Is` is the pack `0, 1, ..., (sizeof...(Views)-1)`. And in view of the fact that standard is very conservative with the `noexcept` specifications in the library specification, this makes `apply` lacks the `noexcept` specification and becomes a non-`noexcept` function. Fortunately, the standard also defines the semantics of `apply` in terms of another equivalent-to *Effects*, this part constitutes the effective `noexcept` specification of `operator*()`.

But if we look at `apply` closely, according to its *Effects* in [\[tuple.apply\]](#):

```
template<class F, class Tuple, size_t... I>
constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, index_sequence<I...>) {
    // exposition only
```

```

    return INVOKE(std::forward<F>(f), get<I>(std::forward<Tuple>(t))...)
}

```

It just simply uses `index_sequence` to expand `get` to extract the elements of `tuple` and then forward them to `INVOKE` together with callable. And since `get` is a `noexcept` function (except for the `subrange`-overload, but the standard does not specify whether `apply` can be applied to `subrange`) and `invoke` is conditional `noexcept`, I think there is no reason not to make `apply` "transparently" become conditional `noexcept`.

In my opinion, `invoke(f, args...)` should be completely equivalent to `apply(f, forward_as_tuple(args...))`, adding `noexcept` to `apply` can easily achieve this and make it more consistent with `invoke`.

Impact on the standard

Since it is a pure change for `apply`, there will be no impact. For the `zip` family, `apply` can "indeed" be used for its implementation. This also allows users to freely add `noexcept` specification to the functions implemented through `apply` in the future.

Implementation experience

This proposal has been implemented by [libstdc++](#) and [libc++](#), and the `libstdc++`'s implementation is based on the premise that `std::get` never throws. [MSVC-STL](#) is consistent with standard and does not add `noexcept` specification to `apply`.

Proposed change

1. Edit 22.4.2 [\[tuple.syn\]](#) as indicated:

```

// [tuple.apply], calling a function with a tuple of arguments
template<class F, class Tuple>
constexpr decltype(auto) apply(F&& f, Tuple&& t) noexcept(see below);

```

2. Edit 22.4.5 [\[tuple.apply\]](#) as indicated:

```

template<class F, class Tuple>
constexpr decltype(auto) apply(F&& f, Tuple&& t) noexcept(see below);

```

-1- *Effects*: Given the exposition-only function:

```

namespace std {
    template<class F, class Tuple, size_t... I>
    constexpr decltype(auto) apply-impl(F&& f, Tuple&& t, index_sequence<I...>) {
        // exposition only
        return INVOKE(std::forward<F>(f), get<I>(std::forward<Tuple>(t))
    ...);    // see [func.require]
    }
}

```

Equivalent to:

```

return apply-impl(std::forward<F>(f), std::forward<Tuple>(t),
    make_index_sequence<tuple_size_v<remove_reference_t<Tuple>>-1>{});

```

-2- Remarks: Let I be the pack $0, 1, \dots, (\text{tuple size } v < \text{remove reference } t < \text{Tuple}> - 1)$. The exception specification is equivalent to: `noexcept(invoke(std::forward<F>(f), __get<I>(std::forward<Tuple>(t)). ...)).`

References

Thomas Köppe. Working Draft, Standard for Programming Language C++. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4901.pdf>

[P2321]

Tim Song. zip. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2321r2.html>

[LIBC++]

apply implementation in libc++. URL: <https://github.com/llvm/llvm-project/blob/d2b0df35afb7184f5a68f67d6ed0c6230688df7f/libcxx/include/tuple#L1576>

[LIBSTDC++]

apply implementation in libstdc++. URL: <https://github.com/llvm/llvm-project/blob/d2b0df35afb7184f5a68f67d6ed0c6230688df7f/libcxx/include/tuple#L1576>

[MSVC-STL]

apply implementation in Microsoft STL. URL: <https://github.com/microsoft/STL/blob/205aed72533849619a6dadbef44eab541a75c549/stl/inc/tuple#L978>